

Lernziele:

- Code-Generierung für Zuweisungen

Zielsetzung

Bei einer Zuweisung `y = 42` kann man Code nach folgendem Schema erzeugen:

- Code erzeugen, um den Wert der rechten Seite in ein Register zu laden. Zum Beispiel in Register `%3`.
- Code erzeugen, um die Adresse der linken Seite (hier die Adresse von `y`) in ein Register zu laden. Zum Beispiel in ein Register `%4`.
- Code erzeugen für eine Store-Operation, die den Inhalt von Register `%3` an die in Register `%4` gespeicherte Adresse schreibt.

Man muss also in der Lage sein, die Adresse einer Variablen in ein Register zu laden. Bei einer globalen Variablen bedeutet das, dass man ein Label in ein Register lädt. Bei einer lokalen Variablen oder einem Funktionsparameter ist die Adresse der Wert des Framepointers (Register `%FP`) plus ein Offset.

Bei einer Zuweisung `y = y + 1` kann man analog verfahren. Der einzige wesentliche Unterschied zu oben ist, dass die Variable hier auch auf der rechten Seite vorkommt und für eine Plus-Operation verwendet wird. Man muss sich also überlegen, wie man den Wert von `y + 1` in ein Register laden kann. Den Code dafür kann man nach folgendem Schema erzeugen:

- Lade den Wert von `y` in ein Register. Zum Beispiel in Register `%3`.
- Lade den Wert `1` in ein Register. Zum Beispiel in Register `%4`.
- Addiere Register `%3` mit Register `%4` und speichere das Ergebnis in Register `%3`.

Hier muss man also den Wert einer Variablen in ein Register laden (oben war es die Adresse der Variable). Das kann man im Allgemeinen mit zwei Instruktionen machen:

- Erst die Adresse der Variablen `y` in ein Register laden. Zum Beispiel in Register `%3`.
- Dann von der in Register `%3` gespeicherten Adresse den Wert mit einer Fetch-Instruktion in ein Register laden. Dabei kann man das Register mit der Adresse überschreiben, also in diesem Fall Register `%3` überschreiben.

Bei lokalen Variablen ist es natürlich möglich, den Wert mit einer einzigen Instruktion zu laden. Zum Beispiel wenn `-8` das Offset für `y` ist mit

```
movq -8(%FP), %3
```

Das kann man aber als Optimierung betrachten und zunächst ignorieren. Obwohl diese Optimierung natürlich wichtig ist, da man lokale Variablen öfter (und insbesondere in Schleifen) verwendet als globale.

Zur Vereinfachung gehen wir davon aus, dass alle Variablen eine Größe von 8 Byte haben und nicht deklariert werden müssen. Außerdem sind zunächst alle Variablen **globale** Variablen. Dann kann man wie folgt festhalten, was der Code-Generator unterstützen muss:

- Adresse einer Variablen in ein Register laden
- Fetch-Instruktionen: Einen Registerinhalt an eine Adresse schreiben, wobei die Adresse ebenfalls in einem Register enthalten ist.
- Store-Instruktionen: Von einer Adresse (die in einem Register steht) einen Wert in ein Register laden.
- Merken, welche Variablen insgesamt verwendet wurden. Damit man am Ende das Datensegment erzeugen kann.

Schritt 1 (Code-Generator erweitern)

Deklariert in `gen.hdr` (am Ende) die folgenden zusätzlichen Funktionen:

```
//-- handle variables
extern fn genLoadAddress(varName: -> char): GenReg;
extern fn genFetch(addr: GenReg): GenReg;
extern fn genStore(val: GenReg, addr: GenReg);
```

Eine erste Version für eine Implementierung gebe ich direkt vor. Hierbei merkt sich der Code-Generator noch nicht, welche Variablen insgesamt verwendet wurden. Und wie gesagt, es wird davon ausgegangen, dass alle Variablen global sind und eine Größe von 8 Bytes haben. Fügt in `gen.abc` (am Ende) folgenden Code hinzu:

```
//-- handle variables
fn genLoadAddress(varName: -> char): GenReg
{
    local reg: GenReg = getReg();
    printf("\tload %s, %%d\n", varName, reg);
    return reg;
}

fn genFetch(addr: GenReg): GenReg
{
    local reg: GenReg = getReg();
    printf("\tmovq (%%d), %%d\n", addr, reg);
    return reg;
}

fn genStore(val: GenReg, addr: GenReg)
{
    printf("\tmovq %%d, (%%d)\n", val, addr);
}
```

Macht euch mit den vorgegebenen Implementierungen vertraut. Zum Beispiel sollte euch klar sein, dass mit `genLoadAddress` einfach der Wert eines Labels in ein freies Register geschrieben wird und dann dieses Register zurückgegeben wird. Auch bei `genFetch` wird ein Register zurückgegeben, das bisher frei war und in das der Wert geladen wurde. Die Funktion `genStore` hat dagegen keinen Rückgabewert, da für diese Operation kein neues Register notwendig ist. Denn der Caller übergibt bereits alles, was hier notwendig ist: das Register mit dem Wert und das Register mit der Adresse.

Im nächsten Schritt werden wir die Datenstruktur für Expression-Trees so erweitern, dass auch für Zuweisungsknoten Code erzeugt wird. Dafür ist es notwendig, dass man nicht nur versteht, was diese Funktionen machen, sondern vor allem, wie man sie verwendet, um für Ausdrücke mit Zuweisungen Code zu erzeugen. Indem ihr jetzt das Testprogramm `xtest_expr.abc` anpasst, sollen also zwei Ziele erreicht werden: Testen, dass diese Funktionen tatsächlich funktionieren und üben, wie man damit für Ausdrücke Code erzeugt.

Folgendes Testprogramm ist vorgegeben und erzeugt Code für `y + 1`. Der Wert des Ausdrucks wird mit der Halt-Instruktion als Exit-Code zurückgegeben.

```
@ "gen.hdr"

fn main()
{
    local r1: GenReg = genLoadAddress("y");
    local r2: GenReg = genFetch(r1);
    local r3: GenReg = genConstant("1");
    local r4: GenReg = genInstr(GEN_ADD, r2, r3);
    genHalt(r4);
    genReleaseReg(r1);
    genReleaseReg(r2);
    genReleaseReg(r3);
    genReleaseReg(r4);
}
```

Aufgaben

1. Übersetzt das Programm natürlich zunächst und führt es aus. Es sollte folgenden Code erzeugen:

```
load y, %1
movq (%1), %2
load 1, %3
add %2, %3, %4
halt %4
```

Unsere Architektur `simple.isa` unterstützt noch keine Fetch- und Store-Instruktionen. Außerdem fehlt in dem Code das Datensegment. Aus diesen Gründen müsst ihr die prinzipielle Korrektheit durch reines Überlegen bestätigen. Die wesentliche Aufgabe für euch ist aber, das Testprogramm so zu ändern, dass insgesamt weniger Register benötigt werden. In obigem Code wird zum Beispiel die Adresse von `y` in Register `%1` geladen. Der Wert von `y` wird dann in ein neues Register `%2` geladen, hier könnte man aber `%1` überschreiben, wenn man vor der Erzeugung der Fetch-Instruktion das Register mit `genReleaseReg(r1)` wieder freigibt. Ändert das Programm so ab, dass es folgenden Code erzeugt:

```
load y, %1
movq (%1), %1
load 1, %2
add %1, %2, %1
halt %1
```

2. Erweitert das Testprogramm, sodass es Code für den Ausdruck

```
y = y + 1
```

erzeugt und der Wert des Ausdrucks als Exit-Code verwendet wird.

3. Register `%1` und Register `%2` werden wir später jeweils für den Stack- und Frame-Pointer verwenden. Ändert den Code-Generator so ab, dass diese zwei Register immer unbenutzt bleiben. Das Testprogramm sollte dann folgenden Code erzeugen:

```
load y, %3
movq (%3), %3
load 1, %4
add %3, %4, %3
load y, %4
movq %3, (%4)
halt %3
```

Schritt 2: Datenstruktur für Expression-Trees anpassen

Seit Session 19 unterstützt unsere Datenstruktur auch Knoten für Zuweisungen. Für einen Expression-Tree kann auch LaTeX-Code erzeugt werden. Allerdings kann für einen Zuweisungsknoten noch kein Assembler-Code erzeugt werden. Im Test `xtest_expr.abc` wurde die Zeile mit dem Aufruf von `loadExpr` auskommentiert. Entfernt nun den Kommentar, so dass ihr folgendes Testprogramm habt:

```

1 @ <stdio.h>
2 @ "expr.h"
3 @ "gen.h"
4
5 fn main()
6 {
7     local expr: -> Expr;
8
9     expr = createBinaryExpr(EXPR_SUB,
10                             createBinaryExpr(EXPR_DIV,
11                                             createIntegerExpr((string)"2"),
12                                             createIntegerExpr((string)"3")),
13                             createIntegerExpr(string"4"));
14     printExprLatexTree(expr);
15     expr = createBinaryExpr(EXPR_ASSIGN,
16                             createIdentifierExpr((string)"x"),
17                             expr);
18     printExprLatexTree(expr);
19     printf("\n");
20
21     genReleaseReg(loadExpr(expr));
22
23     releaseExpr(expr);
24 }
```

Es sollte sich problemlos übersetzen lassen, dann aber bei der Ausführung einen Assertion-Failure produzieren:

```

MCL:s21 lehn$ ./xtest_expr
\item
\[
\begin{forest}
for tree={draw,circle,calign=fixed edge angles}
[ - [ / [ 2 ] [ 3 ] ] [ 4 ] ]
\end{forest}
\]
\item
\[
\begin{forest}
for tree={draw,circle,calign=fixed edge angles}
[ {=} [ x ] [ - [ / [ 2 ] [ 3 ] ] [ 4 ] ] ]
\end{forest}
\]
```

```

expr.abc:87: Assertion '0 && "TODO"' failed.
Abort trap: 6
```

Damit ist eigentlich schon klar, wo man nun etwas anpassen muss. Die Assertion in `expr.abc` in Zeile 87 darf nicht ausgelöst werden. Bei mir sieht die Zeile so aus:

```
case EXPR_IDENTIFIER: assert(0 && "TODO");
```

Hier sollte zunächst die Adresse der Variablen in ein Register geladen werden und dann von dieser Adresse der Wert mit einer Fetch-Instruktion geladen werden.

Aufgabe

1. Ändert den Case ab in:

```
case EXPR_IDENTIFIER:
{
    local addr: GenReg = genLoadAddress(expr->identifizier);
    return genFetch(addr);
}
```

Der Compound-Statement-Block (das, was mit `{` anfängt und mit `}` aufhört) ist notwendig, damit man hier eine lokale Variable anlegen kann. Wenn ihr nach dieser Änderung alles übersetzt und ausführt, wird ein weiterer Assertion-Failure ausgelöst. Um den werden wir uns später kümmern. Schaut euch zunächst an, welchen Assembler-Code davor erzeugt wurde. Relevant sind hier diese zwei Zeilen:

```
load x, %3
movq (%3), %4
```

Die Adresse der Variablen `x` wird in Register `%3` geladen, der Wert von `x` dann in Register `%4`. Ändert den Code in obigem Case so ab, dass Register `%3` mit dem Wert von `x` überschrieben wird.

2. Den neuen Assertion-Failure zu behandeln, wird nun etwas komplexer. Bei mir sieht der Code in der Funktion `loadExpr`, der Binary-Nodes behandelt, aktuell noch so aus:

```
} else if (expr->kind >= EXPR_BINARY && expr->kind < EXPR_BINARY_END) {
    local left: GenReg = loadExpr(expr->left);
    local right: GenReg = loadExpr(expr->right);
    genReleaseReg(left);
    genReleaseReg(right);
    switch (expr->kind) {
        case EXPR_ADD: return genInstr(GEN_ADD, left, right);
        case EXPR_SUB: return genInstr(GEN_SUB, left, right);
        case EXPR_MUL: return genInstr(GEN_MUL, left, right);
        case EXPR_DIV: return genInstr(GEN_DIV, left, right);
        default: assert(0);
    }
}
```

Bisher geht der Code davon aus, dass man immer den Wert der beiden Child-Nodes in Register lädt und dann eine entsprechende Instruktion (zum Beispiel eine Addition) ausführt. Bei einer Zuweisung (wenn `expr->kind` den Wert `EXPR_ASSIGN` hat) soll jedoch Folgendes gemacht werden:

1. Die Adresse des linken Child-Node in ein Register laden.
 2. Den Wert des rechten Child-Node in ein Register laden.
 3. Mit einer Store-Instruktion den Wert des rechten Child-Node an die Adresse des linken Child-Node schreiben.
 4. Das Register mit dem Wert des rechten Child-Node soll dann zurückgegeben werden, denn der Wert einer Zuweisung ist der zugewiesene Wert.
- Damit werden Dinge wie `y = x = 3` unterstützt, wenn die Zuweisungen von rechts nach links ausgewertet werden.

In einem ersten Hack habe ich den Fall einer Zuweisung mit einem If-Statement so behandelt:

```
} else if (expr->kind >= EXPR_BINARY && expr->kind < EXPR_BINARY_END) {
    if (expr->kind == EXPR_ASSIGN) {
        local val: GenReg = loadExpr(expr->right);
        local addr: GenReg = genLoadAddress("TODO");
        genReleaseReg(addr);
        genStore(val, addr);
        return val;
    }
    local left: GenReg = loadExpr(expr->left);
    ...
}
```

Der Fall einer Zuweisung wird also als Spezialfall vorab behandelt.

Und es taucht auch noch ein "TODO" auf. Was noch zu machen ist, wird klar, wenn ihr den Code übersetzt und ausführt. Der Assembler-Code sollte so aussehen:

```
load 2, %3
load 3, %4
div %3, %4, %3
load 4, %4
sub %3, %4, %3
load TODO, %4
movq %3, (%4)
```

Der linke Child-Node in der Zuweisung ist der Identifier `x`. Statt:

```
load TODO, %4
```

sollte deshalb die Instruktion:

```
load x, %4
```

erzeugt werden. Dafür werden wir anschliessend eine Methode `loadExprAddress` implementieren. Vorab könnt ihr aber schon die Zeile:

```
local addr: GenReg = genLoadAddress("TODO");
```

abändern in

```
local addr: GenReg = loadExpr(expr->left);
```

3. In unserem Fall hat nur eine Variable eine Adresse. Deshalb kann man die Implementierung sehr einfach halten. Fügt die Funktion vor der Implementierung von `loadExpr` ein:

```
fn loadExprAddress(expr: ->Expr): GenReg
{
    assert(expr->kind == EXPR_IDENTIFIER);
    return genLoadAddress(expr->identifier);
}
```

Mit der Assertion halten wir fest, dass wir davon ausgehen, dass eine Variable stets durch einen Identifier repräsentiert wird. Insbesondere steht auf der linken Seite einer Zuweisung immer nur ein Identifier. Aktuell ist das richtig. Wenn wir aber später Pointer unterstützen, wird das nicht mehr der Fall sein. Dann kann auch so etwas vorkommen:

```
*x = 42
```

Die Assertion wird uns aber daran erinnern, wo dann etwas erweitert und angepasst werden muss.

Wenn ihr das Testprogramm jetzt übersetzt und ausführt, solltet ihr folgenden Assembler-Code produzieren:

```
load 2, %3
load 3, %4
div %3, %4, %3
load 4, %4
sub %3, %4, %3
load x, %4
movq %3, (%4)
```

Schritt 3 (Compiler anpassen)

Der bisherige Compiler `xtest_abc.abc` unterstützt noch nicht die neue Grammatik aus Session 19, die mehrere Expression-Statements zulässt. Wir machen uns hier das Leben leicht und überschreiben den Compiler mit `xtest_parser.abc`, also mit

```
cp xtest_parser.abc xtest_abc.abc
```

Dann ändert ihr einfach die Zeile:

```
printExprLatexTree(expr);
```

in

```
genReleaseReg(loadExpr(expr));
```

Der Wert einer Expression sollte nach jedem Statement in Register `%3` liegen. Am Ende der Funktion `main` könnt ihr deshalb noch:

```
genHalt(3);
```

hinzufügen. Dann wird der Wert der letzten Expression als Exit-Code verwendet. Zum Beispiel sollte

```
x = 4;
```

```
y = 5;
```

```
z = x*y + x*(x + 4);
```

folgenden Assembler-Code erzeugen:

```
load 4, %3
load x, %4
movq %3, (%4)
load 5, %3
load y, %4
movq %3, (%4)
load x, %3
movq (%3), %3
load y, %4
movq (%4), %4
mul %3, %4, %3
load x, %4
movq (%4), %4
load x, %5
movq (%5), %5
load 4, %6
add %5, %6, %5
mul %4, %5, %4
add %3, %4, %3
load z, %4
movq %3, (%4)
halt %3
```

Schritt 4 (Hardware und Assembler anpassen)

Damit wir den Assembler-Code übersetzen können, muss `simple.isa` um Fetch- und Store-Instruktionen ergänzt werden. Fügt dazu am Anfang folgende Beschreibung für ein Befehlsformat MEM ein:

```
MEM      (OP u 8) (data u 4) (addr u 4) (offset s 16)
```

Und am Ende folgende Beschreibungen für die Fetch- und Store-Instruktionen:

0x23 MEM

```
: movq offset(%addr), %data
```

```
: movq (%addr), %data
```

```
ulm_fetch64(offset, addr, 0, 1, ULM_ZERO_EXT, 8, data);
```

0x24 MEM

```
: movq %data, offset(%addr)
```

```
: movq %data, (%addr)
```

```
ulm_store64(offset, addr, 0, 0, 8, data);
```

Anschließend könnt ihr die neue Hardware und den neuen Assembler mit `ulm-generator --install simple.isa` erzeugen und installieren. Wenn ihr jetzt den Compiler mit `./xtest_abc > foo.s` aufruft, wird der Assembler-Code in `foo.s` geschrieben und ihr könnt diesen mit `ulmas foo.s` übersetzen. Wenn ihr dann aber versucht, mit `ulm a.out` den Maschinencode auszuführen, bekommt ihr die Meldung: `error: 27.1: program contains undefined symbols`. Das liegt daran, dass unser Compiler noch kein Datensegment erzeugt.

Schritt 5: Fake it until you make it

Wenn man es richtig machen will, dann merkt sich der Code-Generator, für welche Variablen `genLoadAddress` aufgerufen wurde, und speichert diese in einer Liste ab. Dann sollte es noch eine Funktion `genDataSegment` geben, die man aufrufen kann, um das Datensegment zu erzeugen.

Die saubere Lösung kann man aber später machen (wir wissen ja wie man eine Liste programmiert) und vorerst einen Hack in den Compiler einbauen, sodass dieser hart verdrahtet am Ende ein Datensegment für drei globale Variablen `x`, `y` und `z` ausgibt. Dann kann man nämlich schon vorab testen, ob das, was wir heute gemacht haben, prinzipiell funktioniert.

Fügt in `xtest_abc.abc` in der Funktion `main` folgenden Code als letzte Zeilen ein:

```
genHalt(3);
printf("\t.align 8\n");
printf("x .quad 8\n");
printf("\t.align 8\n");
printf("y .quad 8\n");
printf("\t.align 8\n");
printf("z .quad 8\n");
```

Jetzt könnt ihr den Compiler zum Beispiel so testen:

```
MCL:s21 lehn$ ./xtest_abc > foo.s
x = 4;
y = 6;
z = x + y;
x*z;
MCL:s21 lehn$ ulmas foo.s
MCL:s21 lehn$ ulm a.out
MCL:s21 lehn$ echo $?
40
```

Etwas Hintergrund: Mit einer Zeile wie

```
printf("x .quad 8\n");
```

wird ein Label `x` angelegt für eine Speicherfläche von 8 Bytes. Die Adresse der Variablen benötigt ein Alignment von 8. Dafür sorgt die mit

```
printf("\t.align 8\n");
```

erzeugte `.align 8` Direktive. Sie weist den Assembler an, dass er notfalls ein paar Padding-Bytes erzeugen soll, sodass das Label auf eine durch 8 teilbare Adresse verweist. Um das nachzuvollziehen, könnt ihr euch den generierten Maschinencode in `a.out` anschauen. Ihr könnt auch eine Zeile für die `Align`-Direktive auskommentieren und dann nochmals den Code anschauen.